Dry Bean Classification ML Project -- Action Plan

Everything You Need To Add/Change (With Complete Code)

Priority-ordered improvements to take your project from 6.5/10 to 9.5/10

TABLE OF CONTENTS

1. [Priority 1: Add Hyperparameter Tuning (GridSearchCV)](#1-add-hyperparameter-tuning)
2. [Priority 2: Add Confusion Matrix & Classification Report](#2-add-confusion-matrix--classification-report)
3. [Priority 3: Create EDA Notebook](#3-create-eda-notebook)
4. [Priority 4: Use Full 13,611 Sample Dataset](#4-use-full-dataset)
5. [Priority 5: Make Feature Engineering Explicit](#5-make-feature-engineering-explicit)
6. [Priority 6: Add SHAP Feature Importance](#6-add-shap-feature-importance)
7. [Priority 7: Add Flask API Endpoint](#7-add-flask-api-endpoint)
8. [Priority 8: Add Learning Curves Plot](#8-add-learning-curves-plot)
9. [Priority 9: Update requirements.txt](#9-update-requirementstxt)
10. [Priority 10: Update README.md](#10-update-readmemd)
11. [Files Changed Summary](#11-files-changed-summary)

1. Add Hyperparameter Tuning

Why: Your current code uses ALL default parameters. This is the #1 thing an interviewer will catch. They'll ask "Did you tune your models?" and right now the answer is no.

Where: `Scripts/benchmark_models.py`

What to change: After the current benchmarking loop, add a GridSearchCV step for the top 3 models.

Complete Modified `benchmark_models.py`

Replace your entire `benchmark_models.py` with this:

```
import json
import os
import argparse
from datetime import datetime

import joblib
import numpy as np
import pandas as pd
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (accuracy_score, f1_score, confusion_matrix,
                             classification_report)
from sklearn.model_selection import (StratifiedKFold, cross_val_score,
                                     train_test_split, GridSearchCV)
from sklearn.naive_bayes import GaussianNB
```

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier

from config_utils import DEFAULT_CONFIG_PATH, load_config


# ■■■ Hyperparameter grids for tuning ■■■■■■■■■■■■■■■■■■■■■■■■■■
PARAM_GRIDS = {
    "svm": {
        "model__C": [0.1, 1, 10, 100],
        "model__gamma": ["scale", "auto", 0.01, 0.001],
    },
    "random_forest": {
        "model__n_estimators": [100, 300, 500],
        "model__max_depth": [None, 10, 20, 30],
        "model__max_features": ["sqrt", "log2"],
    },
    "knn": {
        "model__n_neighbors": [3, 5, 7, 9, 11],
        "model__weights": ["uniform", "distance"],
    },
    "logistic_regression": {
        "model__C": [0.01, 0.1, 1, 10],
        "model__solver": ["lbfgs", "liblinear"],
    },
    "decision_tree": {
        "model__max_depth": [None, 5, 10, 20, 30],
        "model__min_samples_split": [2, 5, 10],
    },
}


def benchmark(config_path: str = DEFAULT_CONFIG_PATH):
    config = load_config(config_path)

    data_path    = config["paths"]["data_path"]
    model_dir    = config["paths"]["model_dir"]
    report_dir   = config["paths"]["report_dir"]
    random_state = int(config["training"]["random_state"])
    cv_splits    = int(config["training"]["cv_splits"])
    test_size    = float(config["training"]["test_size"])
    enabled_models = config["models"]["enabled"]

    # ■■ Load Data ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    if not os.path.exists(data_path):
        raise FileNotFoundError("Run Scripts/data_alignment.py first.")

    df = pd.read_csv(data_path)
    target_col = "Class" if "Class" in df.columns else "Class_Encoded"
    drop_cols = [target_col, "Class", "Class_Encoded", "Unnamed: 0"]
    feature_cols = [c for c in df.columns if c not in drop_cols]

    X = df[feature_cols]
    y = df[target_col]

    # Get class names for confusion matrix
    if "Class" in df.columns:
        class_names = sorted(df["Class"].unique().tolist())
    else:
        class_names = [str(c) for c in sorted(y.unique())]

    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=test_size, random_state=random_state, stratify=y
    )

    cv = StratifiedKFold(n_splits=cv_splits, shuffle=True,
                         random_state=random_state)

    # ■■ Model Registry ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
```

```
model_registry = {
    "logistic_regression": Pipeline([
        ("scaler", StandardScaler()),
        ("model", LogisticRegression(max_iter=2000,
                                     random_state=random_state)),
    ]),
    "knn": Pipeline([
        ("scaler", StandardScaler()),
        ("model", KNeighborsClassifier(n_neighbors=5)),
    ]),
    "decision_tree": Pipeline([
        ("model", DecisionTreeClassifier(random_state=random_state)),
    ]),
    "random_forest": Pipeline([
        ("model", RandomForestClassifier(n_estimators=300,
                                         random_state=random_state,
                                         n_jobs=1)),
    ]),
    "svm": Pipeline([
        ("scaler", StandardScaler()),
        ("model", SVC(kernel="rbf", probability=True)),
    ]),
    "gaussian_nb": Pipeline([
        ("scaler", StandardScaler()),
        ("model", GaussianNB()),
    ]),
}

models = {name: model_registry[name]
          for name in enabled_models if name in model_registry}

if not models:
    raise ValueError("No valid models enabled in config.")

# ■■ Phase 1: Initial Benchmark (all models) ■■■■■■■■■■■■■■■
rows = []
best_name = None
best_model = None
best_holdout_acc = -1.0

print("\n" + "=" * 80)
print("PHASE 1: INITIAL BENCHMARK (default hyperparameters)")
print("=" * 80)

for name, model in models.items():
    cv_scores = cross_val_score(model, X_train, y_train,
                                cv=cv, scoring="accuracy")
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    holdout_acc = accuracy_score(y_test, y_pred)
    macro_f1 = f1_score(y_test, y_pred, average="macro")

    rows.append({
        "model_name": name,
        "cv_accuracy_mean": round(float(cv_scores.mean()), 6),
        "cv_accuracy_std":  round(float(cv_scores.std()), 6),
        "holdout_accuracy": round(float(holdout_acc), 6),
        "holdout_macro_f1": round(float(macro_f1), 6),
        "tuned": False,
    })

    print(f"  {name:&amp;lt;25} CV={cv_scores.mean():.4f}  "
          f"Holdout={holdout_acc:.4f}  F1={macro_f1:.4f}")

    if holdout_acc &amp;gt; best_holdout_acc:
        best_holdout_acc = holdout_acc
        best_name = name
        best_model = model

# ■■ Phase 2: Hyperparameter Tuning (top models) ■■■■■■■■■■■
print("\n" + "=" * 80)
```

```
print("PHASE 2: HYPERPARAMETER TUNING (GridSearchCV)")
print("=" * 80)

tuned_results = {}
for name, model in models.items():
    if name not in PARAM_GRIDS:
        print(f"  {name:&lt;25} No param grid defined, skipping.")
        continue

    print(f"\n  Tuning {name}...")
    grid = GridSearchCV(
        model, PARAM_GRIDS[name],
        cv=cv, scoring="accuracy", n_jobs=-1, verbose=0
    )
    grid.fit(X_train, y_train)

    y_pred = grid.best_estimator_.predict(X_test)
    holdout_acc = accuracy_score(y_test, y_pred)
    macro_f1 = f1_score(y_test, y_pred, average="macro")

    tuned_results[name] = {
        "best_params": grid.best_params_,
        "cv_best_score": grid.best_score_,
        "holdout_accuracy": holdout_acc,
        "macro_f1": macro_f1,
        "estimator": grid.best_estimator_,
    }

    rows.append({
        "model_name": f"{name}_tuned",
        "cv_accuracy_mean": round(float(grid.best_score_), 6),
        "cv_accuracy_std":  0.0,
        "holdout_accuracy": round(float(holdout_acc), 6),
        "holdout_macro_f1": round(float(macro_f1), 6),
        "tuned": True,
    })

    print(f"    Best params:  {grid.best_params_}")
    print(f"    CV accuracy:  {grid.best_score_:.4f}")
    print(f"    Holdout acc:  {holdout_acc:.4f}")
    print(f"    Macro F1:     {macro_f1:.4f}")

    if holdout_acc &gt; best_holdout_acc:
        best_holdout_acc = holdout_acc
        best_name = f"{name}_tuned"
        best_model = grid.best_estimator_

# ■■ Phase 3: Final Evaluation ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
print("\n" + "=" * 80)
print("PHASE 3: FINAL EVALUATION")
print("=" * 80)

y_pred = best_model.predict(X_test)

# Classification Report
report = classification_report(y_test, y_pred,
                               target_names=class_names,
                               output_dict=True)
report_text = classification_report(y_test, y_pred,
                                    target_names=class_names)
print(f"\nBest Model: {best_name}")
print(f"\nClassification Report:\n{report_text}")

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
cm_dict = {"matrix": cm.tolist(), "labels": class_names}

# ■■ Save Everything ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
os.makedirs(model_dir, exist_ok=True)
os.makedirs(report_dir, exist_ok=True)

result_path       = os.path.join(report_dir, "benchmark_results.csv")
```

```
    best_metrics_path = os.path.join(report_dir, "best_model_metrics.json")
    confusion_path    = os.path.join(report_dir, "confusion_matrix.json")
    class_report_path = os.path.join(report_dir, "classification_report.json")
    model_path        = os.path.join(model_dir, "best_model.joblib")
    meta_path         = os.path.join(model_dir, "model_metadata.json")

    result_df = pd.DataFrame(rows).sort_values("holdout_accuracy",
                                               ascending=False)
    result_df.to_csv(result_path, index=False)

    best_row = result_df.iloc[0].to_dict()
    best_metrics = {
        "timestamp_utc": datetime.utcnow().isoformat() + "Z",
        "target_column": str(target_col),
        "best_model_name": best_name,
        "best_model_metrics": {k: v for k, v in best_row.items()
                               if k != "tuned"},
        "feature_columns": feature_cols,
        "class_names": class_names,
        "tuned_params": (tuned_results.get(
            best_name.replace("_tuned", ""), {}).get("best_params", {})),
    }

    with open(best_metrics_path, "w", encoding="utf-8") as f:
        json.dump(best_metrics, f, indent=2)
    with open(confusion_path, "w", encoding="utf-8") as f:
        json.dump(cm_dict, f, indent=2)
    with open(class_report_path, "w", encoding="utf-8") as f:
        json.dump(report, f, indent=2)

    joblib.dump(best_model, model_path)

    with open(meta_path, "w", encoding="utf-8") as f:
        json.dump(best_metrics, f, indent=2)

    print(f"\n[SUCCESS] Model saved to: {model_path}")
    print(f"[SUCCESS] Results saved to: {result_path}")
    print(f"[SUCCESS] Confusion matrix saved to: {confusion_path}")
    print(f"[SUCCESS] Classification report saved to: {class_report_path}")
    print("=" * 80 + "\n")


if __name__ == "__main__":
    parser = argparse.ArgumentParser(
        description="Run model benchmark from YAML config.")
    parser.add_argument("--config", default=DEFAULT_CONFIG_PATH,
                        help="Path to YAML config.")
    args = parser.parse_args()
    benchmark(config_path=args.config)
```

What changed:

- Added `GridSearchCV` import and `PARAM_GRIDS` dictionary
- Added `confusion_matrix` and `classification_report` imports
- Added `probability=True` to SVC (needed for SHAP later)
- Split benchmarking into Phase 1 (defaults) and Phase 2 (tuned)
- Phase 3: generates confusion matrix + classification report
- Saves `confusion_matrix.json` and `classification_report.json`
- Best model selection now considers tuned versions too

2. Add Confusion Matrix & Classification Report

Why: Without a confusion matrix, you can't tell which bean classes your model confuses. This is ML 101 -- every interviewer expects it.

Where: `Scripts/visualize_results.py`

Complete Modified `visualize_results.py`

Replace your entire `visualize_results.py` with this:

```
import json
import os

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from config_utils import load_config


def create_visualizations(config_path: str = "config/benchmark_config.yaml"):
    config = load_config(config_path)
    report_dir = config["paths"]["report_dir"]
    chart_output_path = config["paths"]["chart_output_path"]
    result_path = os.path.join(report_dir, "benchmark_results.csv")
    confusion_path = os.path.join(report_dir, "confusion_matrix.json")

    if not os.path.exists(result_path):
        raise FileNotFoundError(f"Results not found at {result_path}")

    df = pd.read_csv(result_path)

    sns.set_style("whitegrid")
    plt.rcParams['font.family'] = 'DejaVu Sans'

    # ■■ Determine layout: 4 panels or 6 panels ■■■■■■■■■■■■■■■
    has_confusion = os.path.exists(confusion_path)

    if has_confusion:
        fig, axes = plt.subplots(2, 3, figsize=(20, 12))
    else:
        fig, axes = plt.subplots(2, 2, figsize=(14, 10))

    fig.suptitle('Model Benchmark Results - Dry Bean Classification',
                 fontsize=16, fontweight='bold')

    df_sorted = df.sort_values('holdout_accuracy', ascending=False)

    # ■■ Plot 1: Holdout Accuracy Bar Chart ■■■■■■■■■■■■■■■■■■■■
    ax1 = axes[0, 0]
    colors = ['#2ecc71' if '_tuned' in str(n) else '#2E86AB'
              for n in df_sorted['model_name']]
    bars = ax1.bar(range(len(df_sorted)), df_sorted['holdout_accuracy'],
                   color=colors, edgecolor='black', linewidth=1.5)
    ax1.set_xticks(range(len(df_sorted)))
    ax1.set_xticklabels(df_sorted['model_name'], rotation=45, ha='right')
    ax1.set_ylabel('Holdout Accuracy', fontsize=11, fontweight='bold')
    ax1.set_title('Holdout Accuracy by Model', fontsize=12, fontweight='bold')
    y_min = max(0.80, df_sorted['holdout_accuracy'].min() - 0.03)
    ax1.set_ylim(y_min, min(1.0, df_sorted['holdout_accuracy'].max() + 0.02))
    for bar in bars:
        height = bar.get_height()
        ax1.text(bar.get_x() + bar.get_width() / 2., height,
                 f'{height:.3f}', ha='center', va='bottom',
                 fontsize=8, fontweight='bold')

    # ■■ Plot 2: CV vs Holdout Scatter ■■■■■■■■■■■■■■■■■■■■■■■■■
    ax2 = axes[0, 1]
    scatter = ax2.scatter(df['cv_accuracy_mean'], df['holdout_accuracy'],
                          s=200, c=df['holdout_accuracy'], cmap='RdYlGn',
                          edgecolors='black', linewidth=1.5, alpha=0.8)
    ax2.set_xlabel('CV Accuracy (Mean)', fontsize=11, fontweight='bold')
    ax2.set_ylabel('Holdout Accuracy', fontsize=11, fontweight='bold')
    ax2.set_title('CV vs Holdout Accuracy', fontsize=12, fontweight='bold')
    lo = min(df['cv_accuracy_mean'].min(), df['holdout_accuracy'].min()) - 0.02
    hi = max(df['cv_accuracy_mean'].max(), df['holdout_accuracy'].max()) + 0.02
    ax2.plot([lo, hi], [lo, hi], 'k--', alpha=0.5, label='y=x')
```

```
ax2.legend()
for _, row in df.iterrows():
    ax2.annotate(row['model_name'],
                 (row['cv_accuracy_mean'], row['holdout_accuracy']),
                 xytext=(5, 5), textcoords='offset points',
                 fontsize=7, fontweight='bold')

# ■■ Plot 3: Macro F1 Bar Chart ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
if has_confusion:
    ax3 = axes[0, 2]
else:
    ax3 = axes[1, 0]
bars = ax3.bar(range(len(df_sorted)), df_sorted['holdout_macro_f1'],
               color='#A23B72', edgecolor='black', linewidth=1.5)
ax3.set_xticks(range(len(df_sorted)))
ax3.set_xticklabels(df_sorted['model_name'], rotation=45, ha='right')
ax3.set_ylabel('Macro F1 Score', fontsize=11, fontweight='bold')
ax3.set_title('Macro F1 Score by Model', fontsize=12, fontweight='bold')
f1_min = max(0.80, df_sorted['holdout_macro_f1'].min() - 0.03)
ax3.set_ylim(f1_min, min(1.0, df_sorted['holdout_macro_f1'].max() + 0.02))
for bar in bars:
    height = bar.get_height()
    ax3.text(bar.get_x() + bar.get_width() / 2., height,
             f'{height:.4f}', ha='center', va='bottom',
             fontsize=8, fontweight='bold')

# ■■ Plot 4: Confusion Matrix Heatmap ■■■■■■■■■■■■■■■■■■■■■■■
if has_confusion:
    ax4 = axes[1, 0]
    with open(confusion_path, 'r') as f:
        cm_data = json.load(f)
    cm = np.array(cm_data['matrix'])
    labels = cm_data['labels']
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
                xticklabels=labels, yticklabels=labels, ax=ax4)
    ax4.set_xlabel('Predicted', fontsize=11, fontweight='bold')
    ax4.set_ylabel('Actual', fontsize=11, fontweight='bold')
    ax4.set_title('Confusion Matrix (Best Model)', fontsize=12,
                  fontweight='bold')
    ax4.tick_params(axis='x', rotation=45)
    ax4.tick_params(axis='y', rotation=0)

    # ■■ Plot 5: Per-Class Accuracy from Confusion Matrix ■
    ax5 = axes[1, 1]
    per_class_acc = cm.diagonal() / cm.sum(axis=1)
    bars = ax5.barh(labels, per_class_acc, color='#16a085',
                    edgecolor='black', linewidth=1.2)
    ax5.set_xlabel('Accuracy', fontsize=11, fontweight='bold')
    ax5.set_title('Per-Class Accuracy', fontsize=12, fontweight='bold')
    ax5.set_xlim(0.7, 1.05)
    for bar, acc in zip(bars, per_class_acc):
        ax5.text(bar.get_width() + 0.005, bar.get_y() + bar.get_height()/2,
                 f'{acc:.2%}', va='center', fontsize=9, fontweight='bold')

    # ■■ Plot 6: Metrics Table ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    ax6 = axes[1, 2]
else:
    ax6 = axes[1, 1]

ax6.axis('tight')
ax6.axis('off')
table_data = []
for _, row in df_sorted.iterrows():
    table_data.append([
        row['model_name'],
        f"{row['cv_accuracy_mean']:.4f}",
        f"{row['cv_accuracy_std']:.4f}",
        f"{row['holdout_accuracy']:.4f}",
        f"{row['holdout_macro_f1']:.4f}",
    ])
table = ax6.table(
```

```
        cellText=table_data,
        colLabels=['Model', 'CV Mean', 'CV Std', 'Holdout Acc', 'Macro F1'],
        cellLoc='center', loc='center', colColours=['#2E86AB'] * 5)
    table.auto_set_font_size(False)
    table.set_fontsize(8)
    table.scale(1.2, 1.4)
    for i in range(5):
        table[(0, i)].set_facecolor('#2E86AB')
        table[(0, i)].set_text_props(weight='bold', color='white')
    for j in range(5):
        table[(1, j)].set_facecolor('#90EE90')
        table[(1, j)].set_text_props(weight='bold')
    ax6.set_title('Performance Metrics Summary', fontsize=12,
                  fontweight='bold', pad=20)

    plt.tight_layout()
    plt.savefig(chart_output_path, dpi=300, bbox_inches='tight')
    print(f"[SUCCESS] Visualization saved to: {chart_output_path}")
    return chart_output_path


if __name__ == "__main__":
    create_visualizations()
```

What changed:

- Reads `confusion_matrix.json` generated by new benchmark_models.py
- Adds confusion matrix heatmap (Panel 4)
- Adds per-class accuracy bar chart (Panel 5)
- Expands from 4-panel to 6-panel figure
- Color-codes tuned vs default models (green vs blue)

3. Create EDA Notebook

Why: Every ML project needs an EDA notebook. It shows your **thinking process**, not just your code. Interviewers want to see that you explored the data before modeling.

Where: Create `notebooks/EDA.ipynb`

Complete EDA Notebook Code

Create a file `notebooks/EDA.ipynb` and add these cells:

Cell 1 -- Setup:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import LabelEncoder

sns.set_style("whitegrid")
plt.rcParams['figure.figsize'] = (12, 8)
plt.rcParams['font.size'] = 12

df = pd.read_csv('../Data_sets/Dry_Beans_Dataset.csv')
print(f"Dataset shape: {df.shape}")
print(f"Columns: {list(df.columns)}")
df.head()
```

Cell 2 -- Basic Info:

```
print("=" * 60)
print("DATASET INFO")
print("=" * 60)
print(f"\nShape: {df.shape[0]} rows x {df.shape[1]} columns")
print(f"\nData types:\n{df.dtypes}")
print(f"\nMissing values:\n{df.isnull().sum()}")
```

```
print(f"\nStatistical summary:\n{df.describe()}")
```

Cell 3 -- Class Distribution:

```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Bar chart
class_counts = df['Class'].value_counts()
class_counts.plot(kind='bar', ax=axes[0], color='steelblue', edgecolor='black')
axes[0].set_title('Class Distribution (Count)', fontweight='bold')
axes[0].set_ylabel('Count')

# Pie chart
class_counts.plot(kind='pie', ax=axes[1], autopct='%1.1f%%', startangle=90)
axes[1].set_title('Class Distribution (Percentage)', fontweight='bold')
axes[1].set_ylabel('')

plt.tight_layout()
plt.savefig('../reports/eda_class_distribution.png', dpi=150, bbox_inches='tight')
plt.show()

print(f"\nClass counts:\n{class_counts}")
print(f"\nClass balance ratio (max/min): {class_counts.max()/class_counts.min():.2f}")
```

Cell 4 -- Feature Distributions:

```
numeric_cols = df.select_dtypes(include=np.number).columns.tolist()
n_cols = 4
n_rows = (len(numeric_cols) + n_cols - 1) // n_cols

fig, axes = plt.subplots(n_rows, n_cols, figsize=(16, n_rows * 3))
axes = axes.flatten()

for i, col in enumerate(numeric_cols):
    sns.histplot(df[col], kde=True, bins=30, ax=axes[i], color='steelblue')
    axes[i].set_title(col, fontweight='bold', fontsize=10)
    axes[i].axvline(df[col].mean(), color='red', linestyle='--', label='Mean')
    axes[i].axvline(df[col].median(), color='green', linestyle='--', label='Median')
    skew = df[col].skew()
    axes[i].text(0.95, 0.95, f'Skew: {skew:.2f}',
                 transform=axes[i].transAxes, ha='right', va='top',
                 fontsize=8, bbox=dict(boxstyle='round', facecolor='wheat'))

# Hide empty subplots
for j in range(len(numeric_cols), len(axes)):
    axes[j].set_visible(False)

plt.suptitle('Feature Distributions', fontsize=14, fontweight='bold', y=1.02)
plt.tight_layout()
plt.savefig('../reports/eda_distributions.png', dpi=150, bbox_inches='tight')
plt.show()
```

Cell 5 -- Correlation Heatmap:

```
fig, ax = plt.subplots(figsize=(14, 12))
corr = df.select_dtypes(include=np.number).corr()
mask = np.triu(np.ones_like(corr, dtype=bool))
sns.heatmap(corr, mask=mask, annot=True, fmt='.2f', cmap='coolwarm',
            center=0, ax=ax, linewidths=0.5, annot_kws={'size': 7})
ax.set_title('Feature Correlation Matrix', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.savefig('../reports/eda_correlation.png', dpi=150, bbox_inches='tight')
plt.show()

# Find highly correlated pairs
high_corr = []
for i in range(len(corr.columns)):
    for j in range(i+1, len(corr.columns)):
        if abs(corr.iloc[i, j]) &gt; 0.9:
            high_corr.append((corr.columns[i], corr.columns[j],
                              round(corr.iloc[i, j], 3)))
print("\nHighly correlated feature pairs (|r| &gt; 0.9):")
for a, b, r in sorted(high_corr, key=lambda x: abs(x[2]), reverse=True):
```

```
    print(f"  {a} &lt;-&gt; {b}: r = {r}")
```

Cell 6 -- Box Plots (Outlier Detection):

```
fig, axes = plt.subplots(n_rows, n_cols, figsize=(16, n_rows * 3))
axes = axes.flatten()

for i, col in enumerate(numeric_cols):
    sns.boxplot(y=df[col], ax=axes[i], color='lightblue')
    axes[i].set_title(col, fontweight='bold', fontsize=10)

    # Count outliers using IQR method
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    outliers = ((df[col] &lt; Q1 - 1.5*IQR) | (df[col] &gt; Q3 + 1.5*IQR)).sum()
    axes[i].text(0.95, 0.95, f'Outliers: {outliers}',
                 transform=axes[i].transAxes, ha='right', va='top',
                 fontsize=8, bbox=dict(boxstyle='round', facecolor='lightyellow'))

for j in range(len(numeric_cols), len(axes)):
    axes[j].set_visible(False)

plt.suptitle('Outlier Detection (Box Plots)', fontsize=14, fontweight='bold', y=1.02)
plt.tight_layout()
plt.savefig('../reports/eda_boxplots.png', dpi=150, bbox_inches='tight')
plt.show()
```

Cell 7 -- Feature vs Target:

```
top_features = ['Area', 'Perimeter', 'MajorAxisLength', 'MinorAxisLength',
                'roundness', 'Eccentricity']

fig, axes = plt.subplots(2, 3, figsize=(18, 10))
axes = axes.flatten()

for i, feat in enumerate(top_features):
    sns.boxplot(x='Class', y=feat, data=df, ax=axes[i], palette='Set2')
    axes[i].set_title(f'{feat} by Bean Class', fontweight='bold')
    axes[i].tick_params(axis='x', rotation=45)

plt.suptitle('Key Features by Bean Class', fontsize=14, fontweight='bold', y=1.02)
plt.tight_layout()
plt.savefig('../reports/eda_feature_vs_target.png', dpi=150, bbox_inches='tight')
plt.show()
```

Cell 8 -- Pairplot (top features):

```
# Pairplot with top 4 features (more would be too dense)
top4 = ['Area', 'roundness', 'Eccentricity', 'ShapeFactor2', 'Class']
g = sns.pairplot(df[top4], hue='Class', diag_kind='kde',
                 plot_kws={'alpha': 0.5, 's': 15})
g.fig.suptitle('Pairplot of Top Features', fontsize=14, fontweight='bold', y=1.02)
plt.savefig('../reports/eda_pairplot.png', dpi=150, bbox_inches='tight')
plt.show()
```

Cell 9 -- Key Findings Summary:

```
print("=" * 60)
print("EDA KEY FINDINGS")
print("=" * 60)
print("""
1. DATASET: 2,500 samples, 21 features, 7 bean classes
2. NO MISSING VALUES - dataset is clean
3. CLASS BALANCE: Reasonably balanced (no extreme imbalance)
4. HIGHLY CORRELATED FEATURES:
   - Area &amp;amp; Perimeter (r &gt; 0.95) -- expected (larger beans = bigger perimeter)
   - MajorAxisLength &amp;amp; Perimeter (r &gt; 0.95)
   - Consider removing redundant features or using PCA
5. SKEWED FEATURES: Area, Perimeter are right-skewed
   - Log transform may help
6. OUTLIERS: Present in Area, Perimeter, and axis measurements
   - RobustScaler or outlier removal may improve results
7. SEPARABILITY: roundness and Eccentricity show good class separation
```

```
   - These are likely important features for classification
""")
```

4. Use Full Dataset

Why: Your dataset has only 2,500 samples. The UCI Dry Bean dataset has 13,611 samples. Using the full dataset looks more serious and will improve model scores.

Where: Download from UCI ML Repository and replace `Data_sets/Dry_Beans_Dataset.csv`

```
Download URL: https://archive.ics.uci.edu/dataset/602/dry+bean+dataset
File: Dry_Bean_Dataset.xlsx or .csv
Samples: 13,611
Features: 16 (original)
Classes: 7 (SEKER, BARBUNYA, BOMBAY, CALI, DERMASON, HOROZ, SIRA)
```

After downloading, place the file in `Data_sets/Dry_Beans_Dataset.csv` and re-run the pipeline. Your accuracy scores will change (likely improve), and the project looks more substantial.

5. Make Feature Engineering Explicit

Why: Your model metadata mentions engineered features, but the code in `data_alignment.py` doesn't create them. This inconsistency will confuse interviewers.

Where: `Scripts/data_alignment.py`

Complete Modified `data_alignment.py`

```
import os
import numpy as np
import pandas as pd
from config_utils import load_config


def align_data(config_path: str = "config/benchmark_config.yaml"):
    print("LOG: Running data alignment pipeline")
    config = load_config(config_path)

    source_path = "Data_sets/Dry_Beans_Dataset.csv"
    output_path = "Data_sets/train_dataset.csv"

    if not os.path.exists(source_path):
        raise FileNotFoundError(f"Source dataset not found at {source_path}")

    df = pd.read_csv(source_path)

    # Remove index-like export column if present
    if "Unnamed: 0" in df.columns:
        df = df.drop(columns=["Unnamed: 0"])

    if "Class" not in df.columns and "Class_Encoded" not in df.columns:
        raise ValueError("Dataset must include 'Class' or 'Class_Encoded'.")

    # ■■ Feature Engineering ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    print("LOG: Engineering features...")

    # Ratio features (domain-informed)
    df["Area_Perimeter_Ratio"] = df["Area"] / df["Perimeter"]

    # Log transform to reduce skewness
    df["Log_Area"] = np.log1p(df["Area"])

    # Shape complexity measure
    df["Shape_Complexity"] = df["Perimeter"] ** 2 / df["Area"]

    # Axis difference (measures elongation)
```

```
    df["Axis_Difference"] = df["MajorAxisLength"] - df["MinorAxisLength"]

    # Interaction feature
    df["Eccentricity_Roundness"] = df["Eccentricity"] * df["roundness"]

    print(f"LOG: Added 5 engineered features")

    # ■■ Encode Target ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    if "Class_Encoded" not in df.columns and "Class" in df.columns:
        class_labels = sorted(df["Class"].astype(str).unique().tolist())
        mapping = {label: idx for idx, label in enumerate(class_labels)}
        df["Class_Encoded"] = df["Class"].map(mapping)
        print(f"LOG: Encoded {len(class_labels)} classes: {class_labels}")

    df.to_csv(output_path, index=False)
    print(f"LOG: Dataset aligned and saved to: {output_path}")
    print(f"LOG: Rows={len(df)}  Columns={len(df.columns)}")
    print(f"LOG: Feature columns: {[c for c in df.columns if c not in ['Class', 'Class_Encod...


if __name__ == "__main__":
    align_data()
```

What changed:

- Added `numpy` import
- Added 5 engineered features with comments explaining WHY each one
- Added logging for engineered features and class encoding

6. Add SHAP Feature Importance

Why: Shows model explainability. Interviewers love seeing that you understand WHY a model makes predictions, not just THAT it makes predictions.

Where: Create new file `Scripts/explain_model.py`

Complete New File: `Scripts/explain_model.py`

```
"""
Generate SHAP-based feature importance explanations for the best model.
"""
import json
import os

import joblib
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import shap
from config_utils import load_config


def explain_model(config_path: str = "config/benchmark_config.yaml"):
    config = load_config(config_path)
    data_path  = config["paths"]["data_path"]
    model_dir  = config["paths"]["model_dir"]
    report_dir = config["paths"]["report_dir"]

    # Load model and data
    model = joblib.load(os.path.join(model_dir, "best_model.joblib"))
    with open(os.path.join(model_dir, "model_metadata.json"), "r") as f:
        meta = json.load(f)

    df = pd.read_csv(data_path)
    feature_cols = meta["feature_columns"]
    X = df[feature_cols]

    # Use a sample for SHAP (faster)
```

```
    sample_size = min(500, len(X))
    X_sample = X.sample(n=sample_size, random_state=42)

    print(f"Computing SHAP values for {sample_size} samples...")

    # Choose appropriate explainer
    model_name = meta["best_model_name"]
    if "random_forest" in model_name or "decision_tree" in model_name:
        # Tree explainer (fast)
        inner_model = model.named_steps.get("model", model)
        explainer = shap.TreeExplainer(inner_model)
        shap_values = explainer.shap_values(X_sample.values)
    else:
        # Kernel explainer (model-agnostic, slower)
        X_bg = shap.sample(X, 100)
        explainer = shap.KernelExplainer(model.predict, X_bg)
        shap_values = explainer.shap_values(X_sample.values)

    # ■■ Plot 1: Summary Bar Plot ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    plt.figure(figsize=(10, 8))
    if isinstance(shap_values, list):
        # Multi-class: average absolute SHAP across classes
        avg_shap = np.mean([np.abs(sv).mean(axis=0) for sv in shap_values],
                           axis=0)
    else:
        avg_shap = np.abs(shap_values).mean(axis=0)

    feat_importance = pd.Series(avg_shap, index=feature_cols)
    feat_importance = feat_importance.sort_values(ascending=True)

    feat_importance.plot(kind='barh', color='#2E86AB', edgecolor='black')
    plt.xlabel('Mean |SHAP value|', fontweight='bold')
    plt.title(f'Feature Importance (SHAP) - {model_name}',
              fontweight='bold', fontsize=14)
    plt.tight_layout()
    plt.savefig(os.path.join(report_dir, "shap_importance.png"),
                dpi=300, bbox_inches='tight')
    print(f"[SUCCESS] SHAP importance plot saved.")

    # ■■ Save importance ranking ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    importance_dict = feat_importance.sort_values(ascending=False).to_dict()
    with open(os.path.join(report_dir, "shap_importance.json"), "w") as f:
        json.dump(importance_dict, f, indent=2)

    print("\nTop 10 Most Important Features:")
    for feat, val in list(feat_importance.sort_values(ascending=False).items())[:10]:
        print(f"  {feat:&lt;30} SHAP={val:.4f}")


if __name__ == "__main__":
    explain_model()
```

Add to requirements.txt: `shap>=0.42.0`

7. Add Flask API Endpoint

Why: Shows you can deploy a model, not just train one. This is what separates ML engineers from ML students.

Where: Create new file `app.py` in project root

Complete New File: `app.py`

```
"""
Flask API for Dry Bean Classification predictions.
Run: python app.py
Test: curl -X POST http://localhost:5000/predict \
  -H "Content-Type: application/json" \
  -d '{"features": [28395, 610.29, 208.18, ...]}'
```

```
"""
import json
import os

import joblib
import numpy as np
from flask import Flask, jsonify, request

app = Flask(__name__)

MODEL_DIR = "models"
model = joblib.load(os.path.join(MODEL_DIR, "best_model.joblib"))

with open(os.path.join(MODEL_DIR, "model_metadata.json"), "r") as f:
    metadata = json.load(f)

FEATURE_COLS = metadata["feature_columns"]
CLASS_NAMES = metadata.get("class_names", [])


@app.route("/health", methods=["GET"])
def health():
    return jsonify({"status": "healthy", "model": metadata["best_model_name"]})


@app.route("/predict", methods=["POST"])
def predict():
    data = request.get_json()

    if "features" not in data:
        return jsonify({"error": "Missing 'features' key"}), 400

    features = np.array(data["features"]).reshape(1, -1)

    if features.shape[1] != len(FEATURE_COLS):
        return jsonify({
            "error": f"Expected {len(FEATURE_COLS)} features, "
                     f"got {features.shape[1]}"
        }), 400

    prediction = model.predict(features)[0]

    # Get class name if available
    if CLASS_NAMES:
        if isinstance(prediction, (int, np.integer)):
            class_name = CLASS_NAMES[prediction]
        else:
            class_name = str(prediction)
    else:
        class_name = str(prediction)

    result = {
        "prediction": class_name,
        "model": metadata["best_model_name"],
        "features_received": len(FEATURE_COLS),
    }

    # Add probabilities if model supports it
    if hasattr(model, "predict_proba"):
        probas = model.predict_proba(features)[0]
        result["probabilities"] = {
            CLASS_NAMES[i] if CLASS_NAMES else str(i): round(float(p), 4)
            for i, p in enumerate(probas)
        }

    return jsonify(result)


@app.route("/info", methods=["GET"])
def info():
    return jsonify({
```

```
        "model": metadata["best_model_name"],
        "features": FEATURE_COLS,
        "classes": CLASS_NAMES,
        "metrics": metadata.get("best_model_metrics", {}),
    })


if __name__ == "__main__":
    print(f"Loaded model: {metadata['best_model_name']}")
    print(f"Features: {len(FEATURE_COLS)}")
    print(f"Classes: {CLASS_NAMES}")
    app.run(host="0.0.0.0", port=5000, debug=True)
```

Add to requirements.txt: `flask>=2.3.0`

8. Add Learning Curves Plot

Why: Shows whether your model needs more data (high variance) or a better algorithm (high bias). Demonstrates understanding of bias-variance tradeoff.

Where: Add to `Scripts/visualize_results.py` or create separate file

Add to `Scripts/visualize_results.py` (add this function):

```
from sklearn.model_selection import learning_curve, StratifiedKFold

def plot_learning_curves(config_path="config/benchmark_config.yaml"):
    """Generate learning curve plot for the best model."""
    config = load_config(config_path)
    data_path = config["paths"]["data_path"]
    report_dir = config["paths"]["report_dir"]

    df = pd.read_csv(data_path)
    target_col = "Class" if "Class" in df.columns else "Class_Encoded"
    drop_cols = [target_col, "Class", "Class_Encoded", "Unnamed: 0"]
    feature_cols = [c for c in df.columns if c not in drop_cols]
    X = df[feature_cols].values
    y = df[target_col].values

    model = joblib.load(os.path.join(config["paths"]["model_dir"],
                                     "best_model.joblib"))

    cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

    train_sizes, train_scores, val_scores = learning_curve(
        model, X, y, cv=cv,
        train_sizes=np.linspace(0.1, 1.0, 10),
        scoring='accuracy', n_jobs=-1
    )

    train_mean = train_scores.mean(axis=1)
    train_std = train_scores.std(axis=1)
    val_mean = val_scores.mean(axis=1)
    val_std = val_scores.std(axis=1)

    plt.figure(figsize=(10, 6))
    plt.fill_between(train_sizes, train_mean - train_std,
                     train_mean + train_std, alpha=0.1, color='blue')
    plt.fill_between(train_sizes, val_mean - val_std,
                     val_mean + val_std, alpha=0.1, color='green')
    plt.plot(train_sizes, train_mean, 'o-', color='blue', label='Training')
    plt.plot(train_sizes, val_mean, 'o-', color='green', label='Validation')
    plt.xlabel('Training Set Size', fontsize=12, fontweight='bold')
    plt.ylabel('Accuracy', fontsize=12, fontweight='bold')
    plt.title('Learning Curves (Best Model)', fontsize=14, fontweight='bold')
    plt.legend(loc='lower right', fontsize=11)
    plt.grid(True, alpha=0.3)
    plt.tight_layout()
```

```
    plt.savefig(os.path.join(report_dir, 'learning_curves.png'),
                dpi=300, bbox_inches='tight')
    print("[SUCCESS] Learning curves saved.")
```

9. Update requirements.txt

```
# Core ML/Data Science packages
pandas&gt;=1.3.0
numpy&gt;=1.21.0
scikit-learn&gt;=1.0.0
scipy&gt;=1.7.0

# Visualization
matplotlib&gt;=3.5.0
seaborn&gt;=0.12.0

# Model serialization
joblib&gt;=1.3.0

# Configuration
PyYAML&gt;=6.0

# Model Explainability
shap&gt;=0.42.0

# API Deployment
flask&gt;=2.3.0

# Optional
tqdm&gt;=4.66.0
```

10. Update README.md

Add these sections to your README:

```
## Project Improvements

### Hyperparameter Tuning
Models are tuned using GridSearchCV with 5-fold Stratified CV.
Best parameters are saved in `models/model_metadata.json`.

### Model Explainability
SHAP (SHapley Additive exPlanations) provides feature importance
analysis. Run `python Scripts/explain_model.py` to generate.

### API Endpoint
```

python app.py curl http://localhost:5000/predict -X POST \ -H "Content-Type: application/json" \ -d '{"features": [28395, 610.29, ...]}'

```
### EDA Notebook
See `notebooks/EDA.ipynb` for exploratory data analysis including:
- Class distribution analysis
- Feature distributions and skewness
- Correlation heatmap
- Outlier detection
- Feature-target relationships
```

11. Files Changed Summary

Action	File	Description

MODIFY	`Scripts/benchmark_models.py`	Add GridSearchCV, confusion matrix, classification report
MODIFY	`Scripts/data_alignment.py`	Add 5 engineered features
MODIFY	`Scripts/visualize_results.py`	Add confusion matrix heatmap, per-class accuracy, learning c
CREATE	`Scripts/explain_model.py`	SHAP feature importance
CREATE	`app.py`	Flask prediction API
CREATE	`notebooks/EDA.ipynb`	Exploratory Data Analysis
MODIFY	`requirements.txt`	Add shap, flask
MODIFY	`README.md`	Document new features

Estimated time to implement all changes: 4-6 hours

After these changes, your project goes from 6.5/10 to 9.5/10.